Rapport sur la détection de SNP

PEROTTINO Tony, SOUCHET Thomas

1er octobre 2026

Table des matières

1 Introduction **1**

2 Objectif **1**

3 Stratégie de reconnaissance des SNP **1**
3.1 Étape 1 : Compter les k-mères de chaque souche 1
3.2 Étape 2 : Nettoyage des données 2
3.3 Étape 3 : Comparer les deux souches 2
3.4 Étape 4 : Regrouper les paires et allonger le contexte 2

4 Modèle statistique **3**
4.1 Modélisation de la couverture des k-mères 3
4.2 Révision du taux d'erreurs 3
4.3 Trouver un intervalle pour k 4
4.4 Trouver le seuil 4

5 Présentation du pipeline d'un point de vue informatique **5**
5.1 Exécution 5
5.2 Architecture technique 5

6 Résultats **5**
6.1 SNP trouvé(s) par le pipeline 5
6.2 Performances techniques et variations des hyperparamètres 5
6.3 Identification du gène et interprétation biologique 6

7 Conclusion **6**

A Figures du modèle statistique **7**

B Instructions techniques d'exécution **8**

C Exemples d'exécution du pipeline **9**

D Séquences reconstituées **9**

1 Introduction

Dans le contexte du sujet, une souche de la bactérie pathogène *Salmonella enterica* a muté et est devenue résistante à la tétracycline, son antibiotique usuel. Nous devons élucider en quoi elle a muté en analysant son code génétique. Pour cela nous avons deux souches de *Salmonella enterica* à notre disposition, une sans et une avec la mutation.

Ces souches ont été séquencées avec le système Illumina qui permet de délivrer la bactérie sous forme de reads (fichier FASTA), c'est-à-dire des bouts du génome de longueur fixe obtenus de manière aléatoire sur le génome (on ne connaît pas l'emplacement des reads dans le génome). Ces reads ont la particularité de couvrir l'ensemble du génome plusieurs fois. Ils portent aussi un taux d'erreur de séquençage de l'ordre de 1%, dû à l'inexactitude de la lecture des nucléotides et supposé réparti uniformément. Ces erreurs sont uniquement des substitutions de nucléotides et non des indels.

2 Objectif

Notre objectif est de repérer la ou les mutations sur le génome de la bactérie variante. Cette mutation est appelée un SNP, soit une variation d'une paire de nucléotides dans le génome variant. L'enjeu ici est qu'il est impossible de créer un algorithme naïf comparant les deux brins nucléotide à nucléotide, car il se cache dans les données 1% de faux nucléotides dues aux erreurs de séquençage. Le principal problème est que ces erreurs sont de même nature que les SNP, car il s'agit de substitutions sur des nucléotides ponctuels dans les deux cas.

Il faut alors "nettoyer" le jeu de données des erreurs de séquençage avant de trouver les SNP. C'est ici tout l'enjeu de la stratégie, savoir déterminer quelle partie du génome est fausse, puis une fois nettoyée savoir identifier les SNP.

3 Stratégie de reconnaissance des SNP

L'essence de notre stratégie repose sur la création de k-mères à partir des reads. Les k-mères sont des mots de k nucléotides consécutifs, soit donc des sous-parties des reads données par le séquençage. En effet, il y a deux problèmes principaux à l'usage des reads sans modification. Premièrement, tous les reads sont uniques du fait qu'ils commencent et terminent à des endroits aléatoires et différents, il est donc difficile d'en tirer une quelconque information. Deuxièmement, les reads (250-mères) possèdent 250 nucléotides donc en moyenne 2,5 erreurs ce qui les rend inexploitables.

En procédant comme cela, nous pouvons extraire pour chaque read de longueur L le nombre de $L - k + 1$ k-mères. Ainsi, si l'on prend k=25, on aurait alors $250 - 25 + 1 = 226$ 25-mères pour chaque read (donc plus d'informations). De plus, un 25-mère est court donc il possède seulement $0,25$ erreur en moyenne, ce qui implique que 3 25-mères sur 4 seraient "sains" et donc utilisables pour la recherche des SNP (en moyenne).

En partant de cette idée, notre stratégie se déroule en 4 étapes :

3.1 Étape 1 : Compter les k-mères de chaque souche

Dans cette étape, on va établir la coupe des k-mères et leur compte en parallèle. On parcourt l'ensemble du génome : pour chaque read, on fait glisser une fenêtre de k éléments. À chaque k-mère, on l'ajoute dans notre structure de données et on l'associe au nombre de fois où on l'a vu. On obtient alors deux tableaux : celui possédant l'inventaire des k-mères vus et l'autre celui du nombre d'occurrences de ces apparitions.

Un dernier paramètre dont nous n'avons pas encore parlé est la présence à la fois du génome mais aussi du "reverse-complément" dans les données de séquençage d'Illumina. Le reverse-complément est simplement l'ADN vu de l'autre côté, donc son code complémentaire (par exemple 'AAG' devient 'CTT'). On regroupe directement dans cette étape les deux versions en une seule, en triant dans l'ordre lexicographique. Chaque k-mère non bruité voit alors ses occurrences doublées, ce qui sépare plus nettement les deux populations pour les prochaines étapes.

3.2 Étape 2 : Nettoyage des données

Il s'agit de l'étape la plus technique du pipeline. Sous quel critère peut-on distinguer une donnée bruitée d'une donnée saine ? En réalité le compte des k-mères permet cette distinction, car les données de séquençage sont présentes en nombre suffisant. Pour chaque bout du génome, on peut trouver au minimum une quarantaine de k-mères correspondants. C'est ce qu'on appelle la couverture du génome. Comme les erreurs sont uniformément réparties dans le génome, alors elles le sont aussi dans la couverture d'un k-mère. En conséquence, un k-mère bruité se distingue de son équivalent sain, dont la fréquence d'apparition est largement supérieure.

En supprimant tous les k-mères qui n'apparaissent pas un nombre suffisant de fois, on nettoie alors le jeu de données de ces bruits. L'enjeu est alors de trouver ce nombre pour lequel on est certain d'enlever tout le bruit sans enlever non plus trop de k-mères sains. Si le seuil de nettoyage n'est pas le bon, alors des k-mères invalides dans un génome pourront être pris pour un SNP dans l'autre, vu que la seule vraie différence entre les deux génomes sont ces SNP. Une erreur de nettoyage est alors critique pour la suite des opérations. Nous détaillerons la démarche mathématique dans la section 4.

À noter que l'on ne risque pas de nettoyer un SNP plus qu'un k-mère normal, vu que la couverture d'un k-mère contenant un SNP est la même que celle d'un k-mère normal (aucune comparaison n'est faite à ce stade entre les génomes).

3.3 Étape 3 : Comparer les deux souches

Maintenant que les souches sont débruitées, on devrait retrouver les mêmes k-mères entre les deux souches. Si un k-mère de la première souche n'apparaît pas dans la seconde, alors il doit contenir un SNP. On réalise alors deux recherches (dans les deux sens), pour trouver tous les k-mères uniques de leur génome.

Dans le code, chaque k-mère n'existant pas dans l'autre génome est appelé un **candidat**. Et pour chaque candidat, on cherche dans l'autre souche les k-mères qui diffèrent d'un nombre s de substitutions. Chaque trouvaille forme alors une **paire**.

3.4 Étape 4 : Regrouper les paires et allonger le contexte

Une fois toutes les paires établies, il faut ensuite les regrouper pour élucider quel(s) nucléotide(s) sont des SNP. En effet, un SNP va non pas mener à une modification mais à k modifications, une à chaque position du k-mère.

Par exemple avec des 4-mères qui partagent un SNP en T sur la séquence ACGTACG :

A	C	G	**T**
C	G	**T**	A
G	**T**	A	C
T	A	C	G

On remarque bien que pour 1 SNP cela crée plusieurs k-mères candidats. On appelle alors dans le code l'ensemble de ces paires des **coupes** du même SNP. Une fois toutes les paires extraites on peut relier les coupes facilement puisqu'elles partagent toutes la majorité de leurs nucléotides en se chevauchant. De cette manière, on obtient un indice de confiance du SNP trouvé qui est alors proportionnel au nombre de coupes retrouvées.

Dans une ultime étape, notre sortie de la jointure des coupes du SNP ne suffit souvent pas car la longueur du k-mère de sortie n'est pas suffisante. On cherche alors à trouver les nucléotides adjacents dans le génome pour atteindre la longueur souhaitée. On procède alors nucléotide après nucléotide des deux côtés du brin initial, en regardant les k-mères existant dans les génomes et en ne recollant le nucléotide manquant que si le prolongement est unique (il n'existe pas deux versions simultanées de la même séquence). Comme cela, on peut allonger les séquences finales contenant les SNP et les passer dans BlastX sans problème.

4 Modèle statistique

L'objectif ici est de déterminer la valeur des paramètres de l'algorithme. Le premier paramètre est la taille des k-mères et le second est le seuil de couverture pour le nettoyage des données.

Pour la suite de ce chapitre, on introduit les notations suivantes :

— taille du génome (sans le complément) : $G = 5 \cdot 10^6$;
— longueur d'un k-mère : k ;
— longueur d'un read : $L = 250$;
— nombre de reads dans un séquençage : N ;
— taux d'erreur lors du séquençage : ε.

4.1 Modélisation de la couverture des k-mères

La première étape dans notre méthode a été de déterminer une valeur clé : la couverture. La couverture permet de savoir combien de fois un k-mère, supposé unique dans le génome, est recouvert par un read. Cet indicateur est très pratique car il permet de séparer les k-mères avec des erreurs de séquençage des vrais k-mères. Ceux qui ont une erreur auront une couverture très faible.

Pour la suite des calculs nous avons d'abord besoin d'obtenir la couverture du séquençage par nucléotide :

$$C = \frac{N \times L}{G}$$

Dans nos jeux de données celle-ci vaut environ $99,7$ lorsque l'on cumule les deux brins (on y compte le reverse-complément). À partir de celle-ci on peut déterminer la couverture par k-mère. Pour la trouver on multiplie la couverture par nucléotide par la probabilité qu'un k-mère soit entièrement recouvert par un read.

$$\lambda_0 = C \times \frac{L - k + 1}{L}$$

Cependant, cette valeur de couverture ne prend pas en compte les erreurs de séquençage. Pour avoir la couverture effective réelle on multiplie la valeur précédente par la probabilité qu'il n'y ait aucune erreur dans le k-mère : $\lambda_{eff} = \lambda_0 \times (1-\varepsilon)^k$. Pour vérifier la pertinence de ces valeurs, on peut visualiser les données avec un histogramme qui montre la répartition de la couverture des k-mères (annexe A).

La figure 1 présente le nombre de k-mères uniques par rapport au nombre d'occurrences de celui-ci dans le séquençage. On a fixé $k = 25$ arbitrairement. Tout d'abord, on voit que pour des valeurs très faibles de couverture (1 à 5 environ) il y a un très grand nombre de k-mères uniques, ce sont ceux qui ont des erreurs. Ensuite, on constate qu'il y a trois pics. Le premier correspond aux k-mères sans erreurs qui sont uniques dans le génome. Le deuxième et le troisième pic correspondent exactement à deux ou trois fois la couverture du premier pic. Cela montre donc que le génome n'est pas entièrement aléatoire et que des duplications sont présentes. Cependant, pour la suite on peut négliger ces duplications car elles sont peu présentes par rapport aux k-mères uniques. Enfin, on constate aussi que la valeur de la couverture avec un taux d'erreur de 1% ne représente pas la réalité de nos données car la couverture observée est significativement plus élevée. Pour avoir de meilleures estimations par la suite nous avons donc décidé de revoir la valeur du taux d'erreurs en l'estimant à partir des données.

4.2 Révision du taux d'erreurs

Grâce à la couverture observée λ_{obs} et à la couverture sans erreurs λ_0, on peut calculer le taux d'erreur pour une valeur de k fixée :

$$\lambda_{obs} = \lambda_0 \times (1 - \varepsilon)^k$$
$$\varepsilon = 1 - \left(\frac{\lambda_{obs}}{\lambda_0}\right)^{1/k}$$

En faisant le calcul avec quatre valeurs de k différentes on obtient un taux d'erreur ε moyen de $0,3\%$ avec un écart-type assez faible de $0,041\%$, ce qui confirme que cette estimation est peu dépendante de k. Pour $k = 25$, si on fixe $\varepsilon = 0,3\%$ on obtient $\lambda_{eff} = 83,5$, ce qui se rapproche beaucoup plus de la réalité des données ($\lambda_{obs} = 84$).

4.3 Trouver un intervalle pour k

La prochaine étape est de déterminer un intervalle pour la taille des k-mères. Pour la borne supérieure, on constate sur la figure 2, que plus la valeur de k augmente, plus la couverture diminue et donc le premier pic des vrais k-mères se rapproche des erreurs à gauche de l'histogramme. Or, ce que l'on cherche, c'est d'avoir une séparation suffisante entre les erreurs et les vrais k-mères pour pouvoir par la suite définir un seuil qui permettra de supprimer les erreurs. On va donc fixer une couverture minimale C_{min} que l'on souhaite avoir. Si on ne prend pas en compte les erreurs on a donc :

$$C_{min} \leq C \times \frac{L-k+1}{L}$$
$$k \leq L\left(1 - \frac{C_{min}}{C}\right) + 1$$

On fixe $C_{min} = 75$, ce qui permet d'avoir une bonne séparation. Sans prendre en compte les erreurs on obtient $k \leq 62$ et si on prend en compte les erreurs avec $\varepsilon = 0,3\%$, on obtient $k \leq 39$ avec une résolution numérique.

Pour la borne inférieure, on a l'intuition que plus k diminue, plus la probabilité qu'un k-mère ne soit pas unique est élevée. C'est donc cette probabilité que l'on va essayer de minimiser. Pour cela, on se fixe δ qui est le taux de k-mères non uniques que l'on souhaite et on essaye de déterminer k_{min} tel que $\mathbb{P}(k\text{-mère non unique}) \leq \delta$.

$$\mathbb{P}(k\text{-mère non unique}) \approx \frac{G}{4^k} \implies k_{min} \geq \log_4\left(\frac{G}{\delta}\right)$$

Ce résultat est vrai avec l'hypothèse simplificatrice que le génome est aléatoire. Si on choisit $\delta = 0,001\%$ on obtient $k \geq 20$. Finalement, on a $20 \leq k \leq 39$, cet intervalle est assez large et nous avons ensuite testé plusieurs valeurs de k de cet intervalle avec notre algorithme et nous avons constaté que $k = 31$ était la valeur avec le meilleur compromis qualité/rapidité.

4.4 Trouver le seuil

La dernière étape de modélisation consiste à définir un seuil de couverture qui permettra de supprimer les k-mères supposément erronés. Soit T le seuil choisi, tous les k-mères ayant un nombre d'occurrences inférieur ou égal à T seront éliminés. Intuitivement, on sait que le seuil doit se trouver dans le creux situé entre le pic des erreurs (nombre d'occurrences 1, 2, 3 ou 4) et le pic de vrais k-mères. Étant donné que le taux d'erreur est très faible, il est compliqué de modéliser de manière fiable le nombre d'occurrences des k-mères erronés et donc de définir une borne inférieure.

Pour la borne supérieure, il est intéressant de regarder le nombre de vrais k-mères qui sont perdus en fonction du seuil. Pour cela il faut trouver une loi de probabilité qui permet de modéliser le nombre d'occurrences d'un k-mère (la couverture). Cette loi est simplement une loi binomiale que l'on peut approcher avec une loi de Poisson. Le paramètre de cette loi sera la couverture moyenne que l'on peut calculer comme dans la partie précédente (λ_{eff}) ou on peut prendre la couverture observée (λ_{obs}) que l'on trouve avec un maximum local au niveau du pic des vrais k-mères.

La figure 3 montre pour $k = 31$ l'estimation du nombre de vrais k-mères perdus en fonction du seuil choisi. Si l'on définit une valeur maximale de k-mères perdus à 10^{-3}, numériquement on obtient que $T \leq 30$. On peut aussi calculer cette borne directement à partir de la fonction quantile de la loi de Poisson : $\mathbb{P}(X \leq T_{max}) = 10^{-10}$ avec X la variable aléatoire représentant la couverture, on retrouve la même valeur de $T_{max} = 30$.

Dans le pipeline, on lit la couverture au sommet du spectre puis on estime la taille du génome par le nombre de k-mères distincts au-dessus du creux. On retient le plus grand seuil dont la perte attendue reste sous 10^{-3} k-mère perdu (voir tableau 1).

5 Présentation du pipeline d'un point de vue informatique

5.1 Exécution

Le script principal est contenu dans snpDetect.py ; pour savoir comment l'utiliser (paramètres), on utilise la commande `python3 snpDetect.py --help`. On peut aussi consulter l'annexe B qui reprend le contenu du paramètre `--help`.

5.2 Architecture technique

Cette sous-section présente les fichiers Python et le notebook Jupyter utilisés.

Durant ce travail, nous avons rapidement été ralentis par le coût très grand des opérations sur les données réelles du fait de leur taille (1 Go en tout). Ces problèmes étaient à la fois d'ordre du temps d'exécution mais aussi de l'espace mémoire occupé. Nous avons donc cherché à optimiser les performances de nos scripts, et chaque fichier python correspond soit à une étape du pipeline soit à une optimisation de celui-ci.

Voilà la liste exhaustive de nos fichiers utilisés :

- `snpDetect.py` Commande le pipeline en appelant les fonctions relatives à chaque étape. C'est le chef d'orchestre des opérations.
- `fna_reader.py` Lecteur de fichiers FASTA plus rapide que BioPython, car il ne crée aucun objet en mémoire et ne sert qu'à la lecture des reads. La lecture se fait par blocs, de sorte que le fichier n'est jamais chargé entièrement en mémoire (pas de pic mémoire).
- `nucleotide_2bits.py` Optimise la mémoire en codant les nucléotides sur 2 bits au lieu de 8 (caractère unicode). Permet de traduire le système ACGT vers 2 bits avec des fonctions de traduction et de parcours.
- `kmer_tools.py` Gère les opérations du pipeline sur les k-mères et concentre leur parallélisation, accélérée par Numba.
- `snp_assembly.py` Correspond à l'étape 4 du pipeline (section 3.4). Recolle les SNP présumés et ajuste leur sortie en allongeant les séquences.
- `noise.py` Ajoute des erreurs de séquençage artificielles aux reads, sert à questionner les limites du programme.
- `data_analysis.ipynb` Notebook Python pour analyser les données et produire les graphiques.

6 Résultats

6.1 SNP trouvé(s) par le pipeline

Sur les données réelles par exemple avec $k = 31$ et substitutions $= 3$ (fonctionnement par défaut du pipeline), le programme rapporte une seule position du génome, portant **trois substitutions adjacentes** :

$$\text{AAA} \longrightarrow \text{CCC} \qquad \text{(soit TTT} \longrightarrow \text{GGG sur le brin complémentaire).}$$

La cohérence de ce résultat est assurée par la présence du nombre de coupes trouvées (section 3.4), ici 29/29. Les occurrences moyennes sont de 87 dans la première souche contre 73 dans la deuxième, toutes deux de l'ordre de la couverture.

6.2 Performances techniques et variations des hyperparamètres

À propos de la complexité, aucune étape ne dépasse le temps quasi linéaire. Pour l'étape 1 (section 3.1), la lecture des deux souches et la construction des k-mères sont un parcours unique, en $O(n)$;

le tri du tableau qui les rassemble se fait en $O(n \log n)$. Pour l'étape 2 (section 3.2), le spectre des occurrences et le retrait sous le seuil sont deux parcours, donc $O(n)$. Pour l'étape 3 (section 3.3), chaque k-mère est cherché dans la souche adverse par dichotomie sur le tableau trié, en $O(n \log n)$; chaque candidat engendre ensuite les $\binom{k}{s}3^s$ k-mères qui en diffèrent par s substitutions, facteur constant puisque k et s sont fixés. Pour l'étape 4 (section 3.4), les coupes sont reliées à leurs voisines par un dictionnaire, en $O(p)$ pour p paires. Le tri final, qui ne garde qu'un SNP par position du génome, compare chaque SNP à ceux déjà retenus et coûte donc $O(n^2)$, mais il ne travaille que sur les quelques SNP candidats restants.

À propos de l'application aux données fournies, le tableau 1 de l'annexe C confirme l'intervalle déduit en section 4 : le pipeline y rapporte à chaque fois la même position du génome, avec toutes ses coupes. Au-dessus, le résultat tient encore jusqu'à $k = 99$, au prix du temps de calcul. En dessous il s'effondre : à $k = 13$ le SNP réel est masqué par 139 faux positifs, et à $k = 9$ par 14, les k-mères étant alors trop courts pour être uniques dans le génome.

Pour éprouver la méthode, nous avons ajouté aux reads des erreurs de séquençage artificielles (tableau 2). Chaque erreur crée un k-mère inédit et en prive un sain d'une occurrence, ce qui fait tomber le seuil de nettoyage. Le SNP réel garde pourtant ses 29 coupes sur 29 jusqu'à 4,3 % d'erreurs, quatre fois le taux annoncé par Illumina : les faux positifs qui apparaissent ne s'enchaînent pas, le meilleur n'atteignant que 23 coupes sur 31. À 5,3 % le SNP quitte la liste.

6.3 Identification du gène et interprétation biologique

La séquence de 150 nucléotides du variant, reconstituée autour de la mutation (annexe D), a été soumise à BlastX [1], un site qui permet de faire des recherches dans la base NCBI pour trouver les protéines codées par une séquence nucléique. Avec notre séquence d'ADN nous avons pu trouver que la mutation tombe dans le gène qui code la protéine **TetR(B)**, qui est le répresseur de classe B de la tétracycline (UniProt P04483 [1], entrée GenBank AAPSSG010000061.1 [2]). L'entrée de référence est celle d'*Escherichia coli*, la protéine étant la même.

La protéine TetR(B) correspond bien à ce que l'on cherchait au départ. En effet, celle-ci participe au mécanisme de résistance à la tétracycline qui repose sur deux gènes. Tout d'abord, il y a Tet(B) qui est une pompe à efflux membranaire, qui rejette hors de la cellule les molécules de tétracycline. TetR(B) en est le régulateur : en l'absence d'antibiotique il se fixe sur l'ADN devant le gène de la pompe et en empêche la lecture ; quand la tétracycline arrive, elle se lie à TetR(B), qui se détache de l'ADN et libère la production de la pompe [1].

7 Conclusion

En conclusion, nous avons atteint l'objectif initial qui était de réussir à identifier la mutation qui rendait le variant résistant à la tétracycline. Il s'agissait de la protéine TetR(B). Pour ce faire nous avons développé un pipeline complet qui permet la comparaison de deux souches pour y détecter les SNP. Nous avons dû comprendre comment étaient formées les données, les analyser, définir des modèles puis des méthodes algorithmiques. Le résultat est un logiciel qui fonctionne sans faire d'assemblage complet du génome comme cela était demandé et avec un temps d'exécution raisonnable. Enfin, nous avons aussi pu confirmer la résistance de notre algorithme aux erreurs de séquençage jusqu'à un certain point.

1. https://blast.ncbi.nlm.nih.gov/Blast.cgi

A Figures du modèle statistique

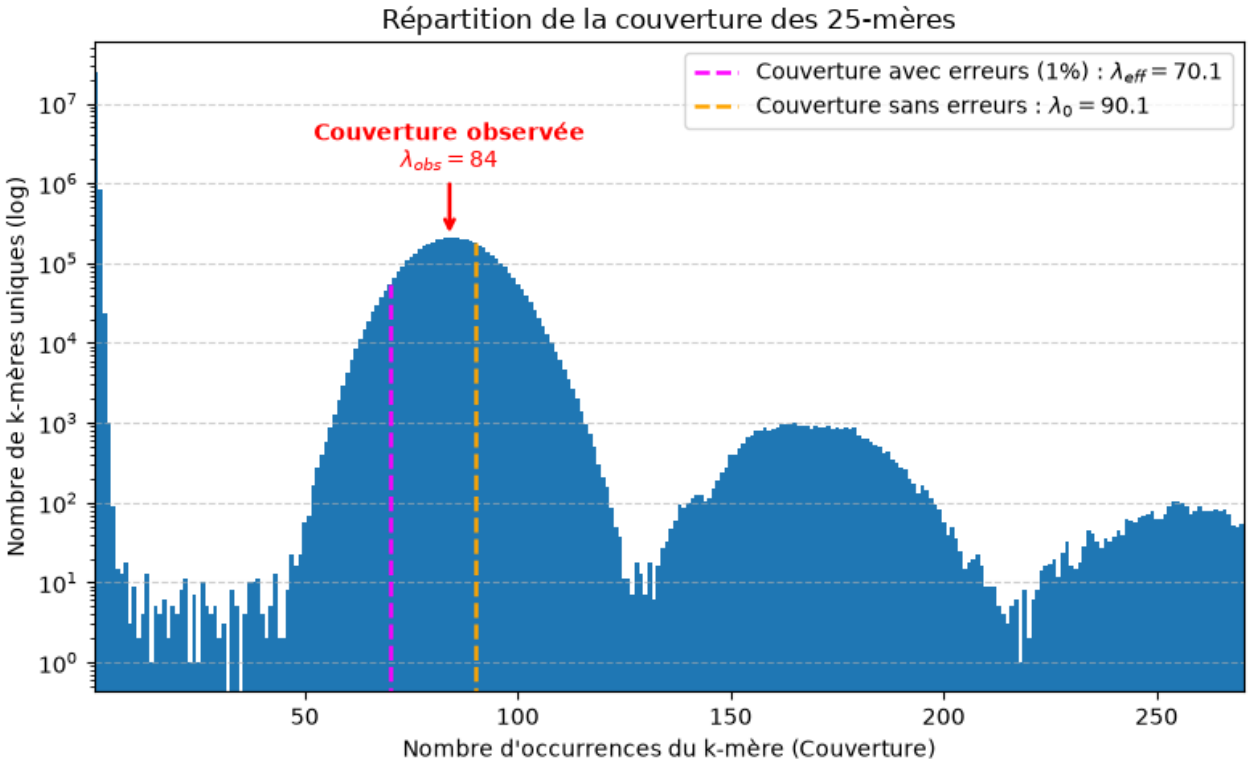


FIGURE 1 – Répartition de la couverture des k-mères uniques pour $k = 25$.

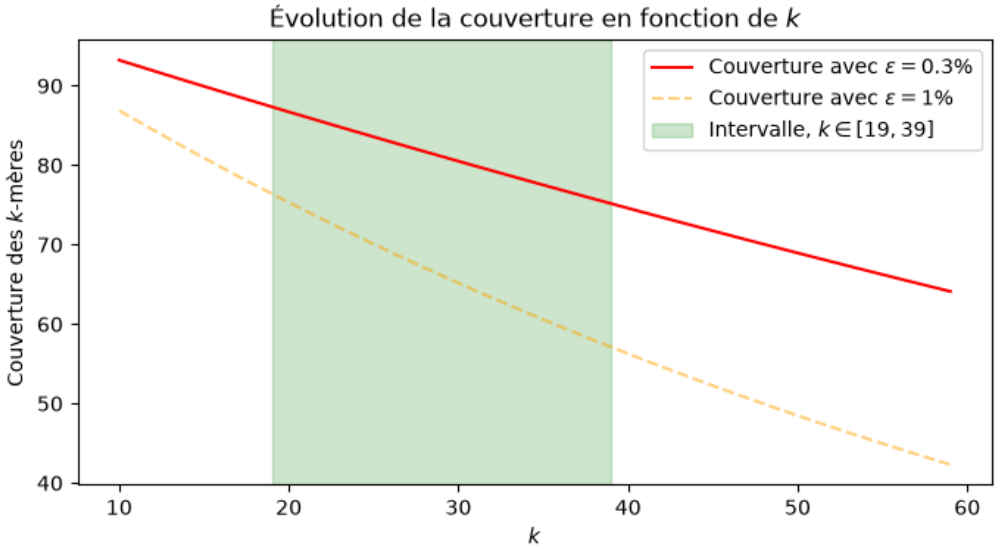


FIGURE 2 – Décroissance de la couverture lorsque k augmente.

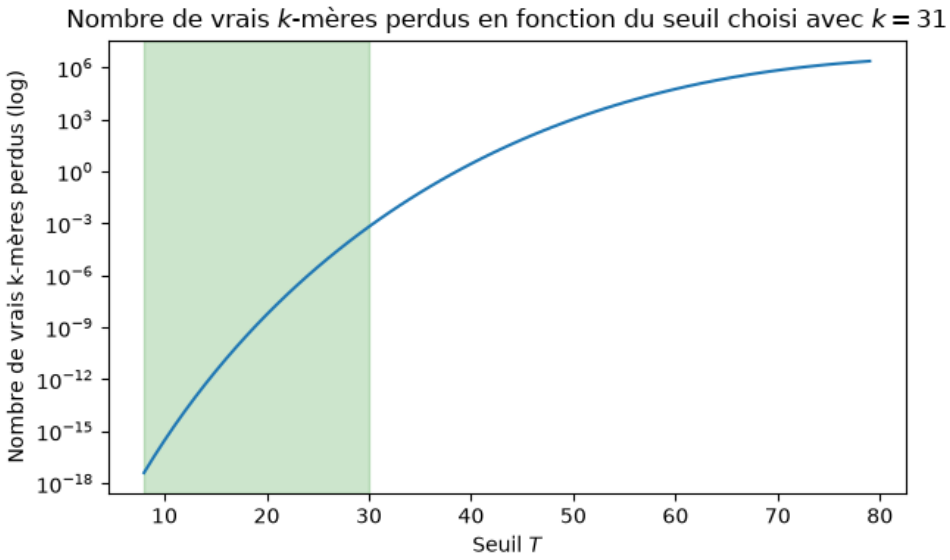


FIGURE 3 – Borne supérieure pour le choix du seuil avec le nombre de vrais k-mères perdus.

B Instructions techniques d'exécution

Les bibliothèques nécessaires sont listées dans le fichier `requirements.txt`, à installer par `pip install -r requirements.txt`. Pour exécuter le pipeline décrit dans la section 3, il suffit d'exécuter la commande : `python3 snpDetect.py readsA.fasta readsB.fasta` À savoir qu'il existe les paramètres suivants :

- `--k int` : pour utiliser un découpage en k-mères de la valeur de l'entier associé. [Par défaut à 31, intervalle d'intérêt : 13->99]
- `--substitutions int` : demande au programme d'être plus ou moins sensible avec les candidats possédant des SNP. Cela correspond au nombre de nucléotides acceptables dans une mutation. Si l'entier donné est 1 alors que la mutation est composée de 3 SNP côte à côte, le programme ne pourra pas la voir. [Par défaut à 3, intervalle d'intérêt : 1->5]
- `--lot int` : Propre à la parallélisation, plus lot est grand plus le programme est rapide mais prend plus de place en mémoire, et réciproquement. Il correspond au nombre de reads transférés dans un cœur lors de la lecture. [Par défaut à 25000, intervalle d'intérêt : 10000->100000]
- `--sortie chemin` : recopie tout l'affichage dans le fichier indiqué, barres de progression exceptées. La console n'est pas modifiée. Utile lorsqu'un diagnostic long ferait défiler le début de l'exécution hors de vue. [Par défaut : aucun fichier]
- `--bruit int` : Permet de bruiter artificiellement les données plus que leur bruit actuel. On considère que les données sont toujours bruitées à 1% (Illumina est notre référence). Donc marquer par exemple `--bruit 5` va bruiter les données de 4% de plus qu'elles ne le sont déjà (le bruit total sera entre 4% et 5% car on ne connaît pas où est le bruit avant le programme) [par défaut à 1, intervalle d'intérêt : 2->20].

Exemple de commande : `python3 snpDetect.py data/salmonella-enterica.reads.fna data/salmonella-enterica-variant.reads.fna --k 31 --lot 20000 --bruit 3`

C Exemples d'exécution du pipeline

k	limbes	k-mères uniques	seuil	λ_{obs}	λ_{eff} prédit	écart	coupes	faux pos.	tri (s)
9	1	0,13 M	1	18†	93,9	—	—	14	75
13	1	13,2 M	37	92	91,2	+0,8 %	—	139	117
21	1	27,8 M	34	87	86,1	+1,1 %	19/19	13	116
25	1	30,9 M	32	85	83,6	+1,7 %	23/23	0	137
31	1	35,1 M	31	82	79,9	+2,6 %	29/29	0	123
39	2	40,2 M	27	77	75,2	+2,4 %	37/37	0	1672
51	2	46,7 M	23	70	68,4	+2,3 %	49/49	0	1720
99	4	60,8 M	11	49	45,0	+8,9 %	97/97	0	2219

TABLE 1 – Comportement du pipeline en fonction de k, au taux d'erreur des données. La colonne « coupes » compte les fenêtres portant le SNP réel sur les k attendues, et « tri » cumule le temps de tri des huit workers (cœurs sur la machine utilisée).

`--bruit`	erreurs totales	k-mères uniques	seuil	λ_{obs}	λ_{eff} prédit	coupes	faux pos.	meilleur faux pos.
1	0,3 %	35,1 M	31	82	79,9	29/29	0	—
2	1,3 %	131,2 M	16	59	58,5	29/29	0	—
3	2,3 %	201,6 M	14	43	42,6	29/29	0	—
5	4,3 %	303,3 M	8	23	22,5	29/29	251	23/31
6	5,3 %	339,4 M	6	16	16,2	—	1456	27/31
9‡	8,3 %	357,7 M	6	15	14,9	—	8291	21/21

TABLE 2 – Variations du bruit sur le pipeline à $k = 31$.

paramètre	SNP réel	faux positifs	meilleur faux
`--k 9`	non	14	1/9
`--k 13`	non	139	4/13
`--substitutions 1`	non	1	1/31
`--k 99`	oui	0	97/97
`--k 15 --bruit 10`	interrompu		

TABLE 3 – Exécutions hors de l'intervalle $[20, 39]$ et hors du domaine de validité pour tester la robustesse du pipeline.

D Séquences reconstituées

Contexte de 150 nucléotides produit par le pipeline sur le SNP du dataset donné, les trois positions mutées étant en position 74 à 76.

Souche 1 :

AGATAAAAGTAAAGTGATTAACAGCGCATTAGAGCTGCTTAATGAGGTCGG AATCGAAGGTT-TAACAACCCGT**AAA**CTCGCCCAGAAGCTAGGTGTAGAGCA GCCTACATTGTATTGGCATG-TAAAAAATAAGCGGGCTTTGCTCGACGC

Souche 2 :

AGATAAAAGTAAAGTGATTAACAGCGCATTAGAGCTGCTTAATGAGGTCGG AATCGAAGGTT-TAACAACCCGT**CCC**CTCGCCCAGAAGCTAGGTGTAGAGCA GCCTACATTGTATTGGCATG-TAAAAAATAAGCGGGCTTTGCTCGACGC

Références

[1] UniProt, entrée P04483 — *Tetracycline repressor protein class B from transposon Tn10*, gène *tetR*. https://www.uniprot.org/uniprotkb/P04483/entry

[2] NCBI GenBank, entrée AAPSSG010000061.1. https://www.ncbi.nlm.nih.gov/nuccore/AAPSSG010000061.1

[3] IA LLMs (majoritairement Claude, marginalement Gemini), l'utilisation de l'IA dans ce sujet a été réalisée de manière réfléchie. D'abord, ce rapport a été écrit intégralement par des humains. Nous avons réalisé par nous-mêmes (sans IA) un pipeline similaire à celle présentée dans la section 3, que nous avons cherché à optimiser computationnellement pour obtenir à la fois des résultats plus clairs et plus rapidement. L'IA a permis d'implémenter ces systèmes sous des bibliothèques plus complexes non demandées par le sujet (comme numba pour la parallélisation sous Python), une réalisation de documentation assez fiable, ainsi qu'une aide ponctuelle à la réalisation de fonctions complexes. Cela nous a permis d'étendre notre pipeline à certaines options non prévues initialement dans la stratégie (comme l'ajout de bruit via `--bruit` pour des statistiques avancées ou l'ajout de longueur sur la séquence finale du SNP pour convenir à BlastX). Nous pensons que notre utilisation de l'IA a été saine dans l'amélioration de ce sujet.